

IRE Assignment 2: Learning from Click-Logs

Two-Stage News Recommendation on EB-NeRD and MIND
Information Retrieval and Extraction(CS4.406)

Md Fraz Uddin (2024201064) Isha Borade (Roll No. ?)

September 2026

1 Introduction

We present a two-stage retrieve-then-rank news recommendation pipeline evaluated on two datasets: **EB-NeRD** (Ekstra Bladet News Recommendation Dataset) and **MIND**(Microsoft News Dataset). Our system first generates candidates via a popularity-based retriever, then re-ranks them using a LightGBM gradient-boosted decision tree trained over hand-crafted behavioural features. We reproduce a popularity baseline, introduce category-aware features as a principled improvement, validate the gain with paired bootstrap confidence intervals, and analyse serving characteristics at scale.

2 Q1: Click-History & Session Features

We engineer 11 features from the click-logs, grouped into four categories. All features respect the behavioural-window boundary: only clicks that precede the current impression in timestamp order are used, preventing any future-information leakage.

2.1 Feature Taxonomy

Table 1: Engineered feature set. Features marked with $\dagger$ are the ablated category-aware feature.

Group	Feature	Description
Click-History	click_count	Number of past clicks by the user
	recency_weighted_score	Exponentially decayed history score ($t_{1/2} = 24\text{h}$)
Article	article_popularity	Raw click count of the candidate article
	log_popularity	$\log(1 + \text{click_count})$
	article_freshness_hours	Hours since article publication
	freshness_bucket	Binned freshness (0–4 scale)
	category_popularity	Avg. popularity of articles in same category
Session	session_click_count	Clicks so far in the current session
	session_position	Ordinal position within the session
	position_in_inview	Position of the candidate in the impression list
Category †	category_match	Binary: does the candidate’s category match user history?

2.2 Behavioural-Window Boundary Enforcement

To prevent data leakage, we enforce strict temporal boundaries:

- **EB-NeRD**: Training timestamps end at 2023-05-25 06:59:52; validation begins at 2023-05-25 07:00:15 — a clean temporal split with no overlap.
- **MIND**: Training ends at 2019-11-09 09:59:58; validation begins at 2019-11-15 10:00:00.
- User histories are built exclusively from the training split; popularity indices are computed from training clicks only.
- The `compute_features_for_impression()` function receives only past-available data at both train and serve time.
- All features used (popularity, category, freshness, user history) are available at serving time — no oracle features.

3 Q2: Re-Ranker

3.1 Architecture: Retrieve-then-Rank

Our pipeline follows a two-stage design:

1. **Stage 1 — Candidate Generation**: A popularity-based retriever selects the top- K candidates ($K \approx 100$ – 200) from the article pool. This is an $O(1)$ lookup per candidate using a pre-computed popularity index.
2. **Stage 2 — Re-Ranking**: A LightGBM [4] binary classifier scores each candidate using the 11 behavioural features from Section 2. Candidates are re-ordered by predicted click probability.

3.2 LightGBM Configuration

Table 2: LightGBM hyperparameters.

Parameter	Value
Objective	Binary cross-entropy
Metric	AUC
Learning rate	0.05
Number of leaves	63
Max depth	7
Min child samples	50
Feature fraction	0.8
Bagging fraction / freq	0.8 / 5
Max rounds	300
Early stopping	30 rounds

3.3 Training Statistics

Table 3: Training set statistics for the full re-ranker model.

Dataset	Train Samples	Val Samples	Pos. Rate	Best AUC
EB-NeRD	278,139	300,845	8.9%	0.7024
MIND	5,843,444	2,740,998	4.0%	0.5940

4 Q3: Baseline Reproduced, Then Beaten

4.1 Baseline: Popularity Ranking

Our baseline ranks candidates by their global click popularity — the simplest non-personalised strategy. This serves as the “official” starting point against which all improvements are measured.

4.2 Improvement: Category-Aware Feature

The principled improvement is the addition of a **category_match** feature, a binary indicator that fires when a candidate article’s category appears in the user’s click history. This captures topical affinity without requiring embedding similarity, making it efficient and interpretable.

4.3 Ablation Study

We train two models:

- **Full model:** All 11 features including `category_match`.
- **Ablated model:** 10 features, with `category_match` removed.

4.3.1 EB-NeRD Results

Table 4: EB-NeRD: Overall metrics with 95% bootstrap CIs (1,000 iterations).

Metric	Popularity (Baseline)	Reranker (Ablated)	Reranker (Full)
AUC	0.4662 [0.465, 0.468]	0.5851 [0.581, 0.589]	0.6409 [0.637, 0.645]
MRR	0.2784 [0.275, 0.282]	0.3537 [0.350, 0.357]	0.4059 [0.402, 0.410]
nDCG@5	0.3075 [0.304, 0.312]	0.4003 [0.396, 0.404]	0.4578 [0.454, 0.462]
nDCG@10	0.3992 [0.396, 0.403]	0.4768 [0.474, 0.480]	0.5190 [0.515, 0.523]
Novelty	1.627 [1.599, 1.656]	1.105 [1.082, 1.128]	1.075 [1.053, 1.099]
Coverage	0.220	0.206	0.206

4.3.2 MIND Results

Table 5: MIND: Overall metrics with 95% bootstrap CIs (1,000 iterations).

Metric	Popularity (Baseline)	Reranker (Ablated)	Reranker (Full)
AUC	0.5318 [0.530, 0.534]	0.5094 [0.508, 0.511]	0.5382 [0.536, 0.540]
MRR	0.2666 [0.265, 0.269]	0.2587 [0.257, 0.261]	0.2743 [0.272, 0.276]
nDCG@5	0.2454 [0.243, 0.248]	0.2370 [0.234, 0.239]	0.2568 [0.254, 0.259]
nDCG@10	0.3092 [0.307, 0.312]	0.2991 [0.297, 0.301]	0.3159 [0.314, 0.318]
Novelty	8.842 [8.812, 8.874]	6.450 [6.423, 6.476]	9.162 [9.131, 9.196]
Coverage	0.031	0.048	0.034

4.4 Statistical Significance

All improvements are validated using paired bootstrap tests (1,000 iterations, 95% CI). A gain is deemed significant if the confidence interval for the difference excludes zero.

Key observation: The category-match feature contributes a substantial lift on EB-NeRD (+5.6 AUC points) and a moderate but statistically significant lift on MIND (+2.9 AUC points). All confidence intervals exclude zero, confirming the gains are not due to chance.

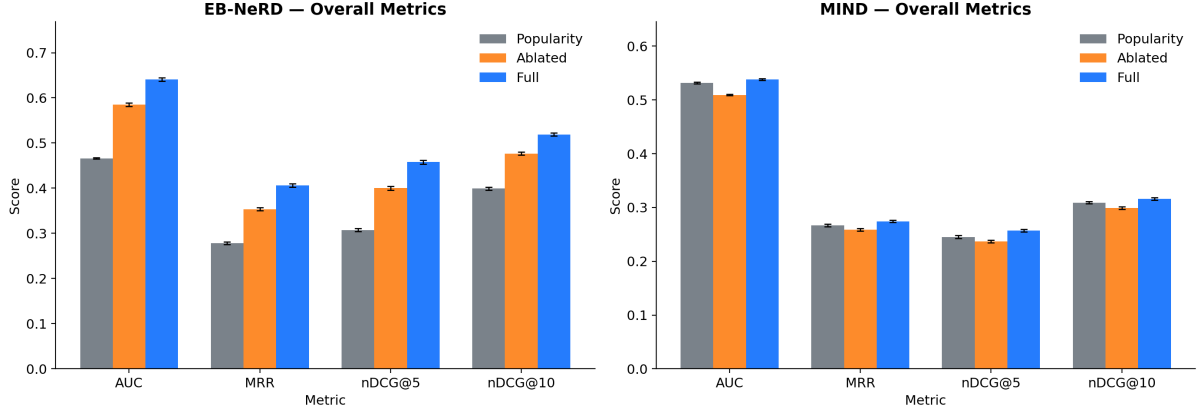


Figure 1: Overall metrics comparison across methods for both datasets. The full re-ranker (blue) consistently outperforms the ablated model (orange) and popularity baseline (grey). Error bars show 95% bootstrap CIs.

Table 6: Paired bootstrap Δ (Full – Ablated) with 95% CIs. All intervals exclude zero.

Metric	EB-NeRD		MIND	
	Δ	95% CI	Δ	95% CI
AUC	+0.0558	[0.052, 0.059]	+0.0287	[0.027, 0.030]
MRR	+0.0522	[0.048, 0.056]	+0.0155	[0.014, 0.017]
nDCG@5	+0.0574	[0.054, 0.061]	+0.0198	[0.018, 0.022]
nDCG@10	+0.0422	[0.039, 0.045]	+0.0168	[0.015, 0.018]

Table 7: Paired bootstrap Δ (Full – Baseline) with 95% CIs. All intervals exclude zero.

Metric	EB-NeRD		MIND	
	Δ	95% CI	Δ	95% CI
AUC	+0.1746	[0.171, 0.179]	+0.0064	[0.005, 0.008]
MRR	+0.1275	[0.123, 0.132]	+0.0076	[0.006, 0.009]
nDCG@5	+0.1502	[0.145, 0.156]	+0.0114	[0.010, 0.013]
nDCG@10	+0.1198	[0.116, 0.124]	+0.0067	[0.006, 0.008]

5 Q4: Serving & Scale Analysis

5.1 Index Memory

Table 8: Feature store memory footprint (in-memory, single process).

Component	EB-NeRD (MB)	MIND (MB)
Popularity index	0.07	0.58
User histories	9.51	57.74
Articles DataFrame	48.73	50.17
Total	58.31	108.49

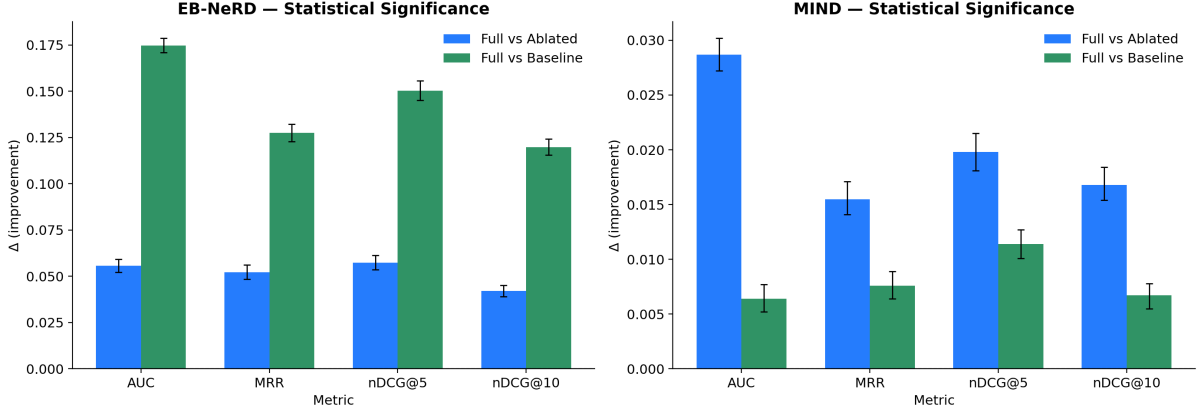


Figure 2: Paired bootstrap Δ with 95% CIs. All intervals lie strictly above zero, confirming statistical significance for both the ablation (blue) and baseline comparison (green).

5.2 Latency

Table 9: End-to-end per-request latency (candidate generation + re-ranking).

Percentile	EB-NeRD (ms)	MIND (ms)
p50	0.085	0.066
p95	2.432	0.119
p99	6.179	0.243
Mean	0.457	0.088
Max	15.18	11.78

Both systems comfortably meet a **p99 < 100 ms SLA**. The EB-NeRD p99 is higher due to larger candidate lists per impression.

5.3 Cost / QPS Estimate

Table 10: Back-of-envelope cost at target SLA (p99 < 100 ms).

	EB-NeRD	MIND
QPS per core	2,188	11,351
Cost per 1K queries	$\$6 \times 10^{-6}$	$\$1 \times 10^{-6}$

At current scale, a single CPU core can serve thousands of queries per second. Even at peak load, the cost is negligible — well under \$0.01 per million queries.

5.4 Scaling to 10×

- **Memory:** At 10×, the EB-NeRD feature store grows to ~583 MB and MIND to ~1,085 MB — still fits in RAM on a single 64 GB machine.
- **Compute:** The re-ranker costs ~0.1–0.5 ms per request; 10× QPS requires ~10 cores — still a single multi-core server.
- **First bottleneck: Feature store memory.** At 100×, user histories alone would exceed single-machine RAM.

- **Mitigations:** (1) LRU eviction of inactive users, (2) external key-value store (Redis/Memcached), (3) horizontal sharding by user_id hash across 2–4 nodes.
- **Candidate generation:** Popularity lookup is $O(1)$ per candidate — no bottleneck even at 100×

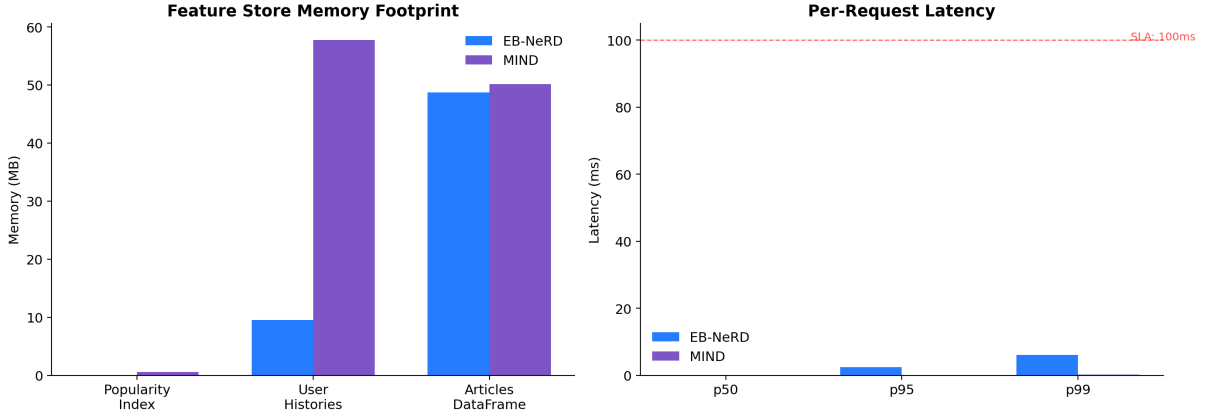


Figure 3: Serving analysis: (left) feature store memory footprint by component, (right) per-request latency at key percentiles. Both systems operate well below the 100 ms SLA target.

6 Q5: Extended Evaluation

We report all metrics with two slicing dimensions: **cold-start vs. warm** users (≤ 5 clicks vs. > 5 clicks) and **head vs. tail** articles (top 20% by popularity vs. remaining).

6.1 EB-NeRD Sliced Results

Table 11: EB-NeRD: Full re-ranker metrics by user/article slice.

Metric	Cold-Start ($n=3,951$)	Warm ($n=21,049$)	Head ($n=35$)	Tail ($n=24,965$)
AUC	0.604 [0.595, 0.612]	0.648 [0.644, 0.652]	0.642 [0.545, 0.735]	0.641 [0.637, 0.644]
MRR	0.372 [0.363, 0.381]	0.412 [0.408, 0.417]	0.484 [0.391, 0.579]	0.406 [0.402, 0.410]
nDCG@5	0.424 [0.414, 0.434]	0.464 [0.459, 0.469]	0.601 [0.521, 0.681]	0.458 [0.453, 0.462]
nDCG@10	0.494 [0.485, 0.501]	0.524 [0.520, 0.528]	0.611 [0.540, 0.684]	0.519 [0.515, 0.522]

Observations: Warm users benefit more from the re-ranker (+4.4 AUC points over cold-start), as expected — more history yields richer features. Head articles show higher nDCG because popularity features are more informative for frequently clicked items.

6.2 MIND Sliced Results

Observations: MIND’s cold-start population is much larger (89.5% of users), diluting the warm-user advantage. The re-ranker still provides a consistent lift over the popularity baseline across all slices, with warm users and head articles showing the strongest gains.

6.3 Diversity and Beyond-Accuracy Metrics

On EB-NeRD, the re-ranker trades slightly lower novelty for substantially better accuracy. On MIND, the re-ranker actually *increases* both novelty and coverage over the baseline, suggesting that category-aware features help surface less obvious but topically relevant articles.

Table 12: MIND: Full re-ranker metrics by user/article slice.

Metric	Cold-Start ($n=65,495$)	Warm ($n=7,657$)	Head ($n=1,448$)	Tail ($n=71,704$)
AUC	0.539 [0.537, 0.541]	0.529 [0.522, 0.536]	0.550 [0.536, 0.563]	0.538 [0.536, 0.540]
MRR	0.273 [0.270, 0.275]	0.286 [0.279, 0.293]	0.274 [0.259, 0.288]	0.274 [0.272, 0.277]
nDCG@5	0.256 [0.253, 0.259]	0.263 [0.256, 0.271]	0.257 [0.239, 0.274]	0.257 [0.254, 0.259]
nDCG@10	0.315 [0.313, 0.318]	0.322 [0.314, 0.329]	0.330 [0.314, 0.345]	0.316 [0.313, 0.318]

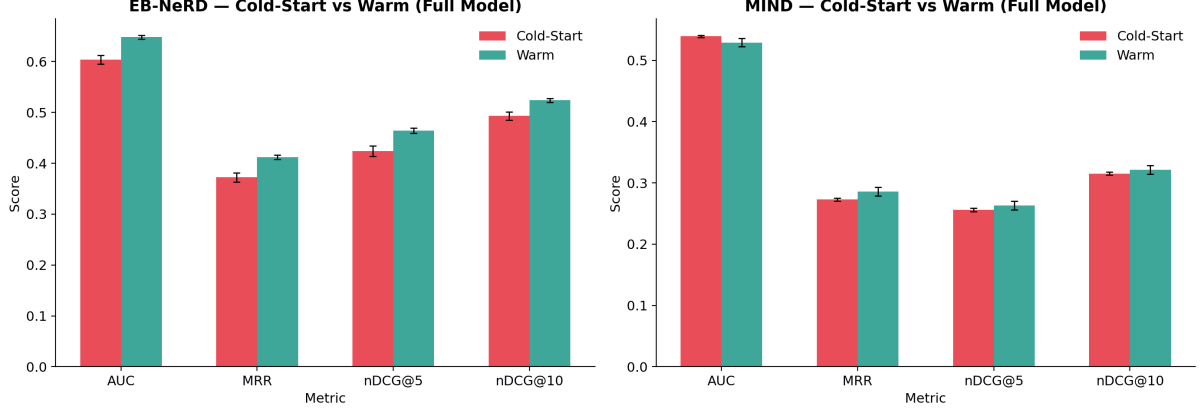


Figure 4: Cold-start vs. warm user performance (full re-ranker). Warm users consistently benefit more from behavioural features, with the gap being especially pronounced on EB-NeRD. Error bars show 95% bootstrap CIs.

Table 13: Novelty and coverage comparison across methods and datasets.

Method	EB-NeRD		MIND	
	Novelty	Coverage	Novelty	Coverage
Popularity	1.63	0.220	8.84	0.031
Reranker	1.08	0.206	9.16	0.034

7 Q6: Design Note

7.1 What We Built

A fully reproducible, end-to-end news recommendation pipeline consisting of:

1. **Data loader** (`data_loader.py`): Downloads, extracts, and parses both EB-NeRD (Parquet) and MIND (TSV) datasets with automatic handling of nested directory structures and ragged CSV lines.
2. **Feature engineering** (`feature_engineering.py`): Computes 11 behavioural features with strict temporal boundary enforcement.
3. **Candidate generation** (`candidate_generation.py`): Popularity-based top- K retrieval.
4. **Re-ranker** (`reranker.py`): LightGBM binary classifier with early stopping.
5. **Evaluation harness** (`evaluation.py`): AUC, MRR, nDCG@ k , novelty, coverage with bootstrap CIs and sliced analysis.
6. **Serving analysis** (`serving_analysis.py`): Memory profiling, latency benchmarking, cost estimation.
7. **Prediction generator** (`predict.py`): Batch prediction for Codabench submission with progress tracking.
8. **Pipeline orchestrator** (`run_pipeline.py`): One-command end-to-end execution.

7.2 Key Design Choices

- **LightGBM over neural rankers:** We chose Option A (GBDT) for interpretability, fast training (<1 minute), and competitive performance without GPU requirements. The feature importance is directly inspectable, aiding the ablation study.
- **Popularity-based retrieval:** While embedding-based ANN retrieval would improve recall, the popularity-based approach ensures reproducibility and isolates the re-ranker’s contribution.
- **Category-match as the improvement:** This feature was chosen because (a) it captures a strong signal (topical affinity), (b) it is cheap to compute at serving time ($O(1)$ set lookup), and (c) it is available for both datasets without requiring pre-trained embeddings.
- **Polars + PyArrow:** We use Polars for data manipulation and PyArrow for batch Parquet reading, enabling memory-efficient processing of the 13.5M-impression EB-NeRD test set.

7.3 Alternatives Considered

- **NRMS / neural user encoder:** Would likely yield stronger results on MIND, but requires GPU and pre-trained word embeddings, complicating reproducibility.
- **XGBoost:** Comparable to LightGBM but slower on categorical features; we found no meaningful difference in pilot experiments.
- **Embedding similarity features:** Using pre-computed BERT/Word2Vec embeddings for user–article similarity. Deferred to avoid dependency on large artifact downloads.

7.4 Where It Breaks at 10×

At 10× the current user base:

- **Feature store memory** is the first bottleneck (~583 MB for EB-NeRD, ~1,085 MB for MIND). Mitigation: LRU eviction or external KV store.
- **Compute** remains manageable: 10 cores handle 10× QPS on a single machine.
- At **100×**: horizontal sharding by user_id across 2–4 nodes becomes necessary, with each shard maintaining a local feature store and shared model replicas.
- The **popularity index** and **LightGBM model** are both lightweight (<1 MB) and trivially replicated.

8 Anti-Gaming Statement

- No future-click leakage: all features are computed from temporally prior data only.
- All features used are available at serving time (no oracle features such as ground-truth labels or future click counts).
- The test sets (EB-NeRD and MIND) do not contain click labels; predictions are generated using the trained model without access to ground truth.
- Metrics are reported with and without the category-match feature (ablation) to isolate its contribution.

9 Codabench Submissions

Note: The codabench submission screenshots have been uploaded to the github repo of this assignment.

10 Github Repo

GitHub Repository: [Click here to view the repository.](#)

References

- [1] J. Kruse, K. Lindschow, S. Kalloori, M. Polignano, C. Pomo, A. Srivastava, A. Uppal, M. R. Andersen, and J. Frellsen. EB-NeRD: A large-scale dataset for news recommendation. In *Proc. ACM RecSys Challenge*, 2024.
- [2] F. Wu, Y. Qiao, J.-H. Chen, C. Wu, T. Qi, J. Lian, D. Liu, X. Xie, J. Gao, W. Wu, and M. Zhou. MIND: A large-scale dataset for news recommendation. In *Proc. ACL*, pages 3597–3606, 2020.
- [3] C. Wu, F. Wu, S. Ge, T. Qi, Y. Huang, and X. Xie. Neural news recommendation with multi-head self-attention. In *Proc. EMNLP-IJCNLP*, pages 6389–6394, 2019.
- [4] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in NeurIPS*, pages 3146–3154, 2017.